FBK K8s Extension meets TUB Energy Aware Algorithm - Energy Aware Scheduler

This document briefly report the architecture and the assumptions made for the integration of the `FBK K8s extension` with the `TUB energy-aware algorithm` creating a system able to scheduler workload in a K8s cluster taking into account the energy consumption and the energy mix of the nodes where the workload should run. We called this system `K8s Energy Aware Scheduler`.

What is FBK K8s Extension

FBK K8s Extension is composed by three components:

- `K8s custom scheduler plugin` for handling the custom scores provided by an external algorithm
- `Custom kubectl plugin` so as to manage to deployment of a pod according to the scheduling scores and time output from an external algorithm
- `grpc IDL interface` between `kubectl plugin` and the external algorithm

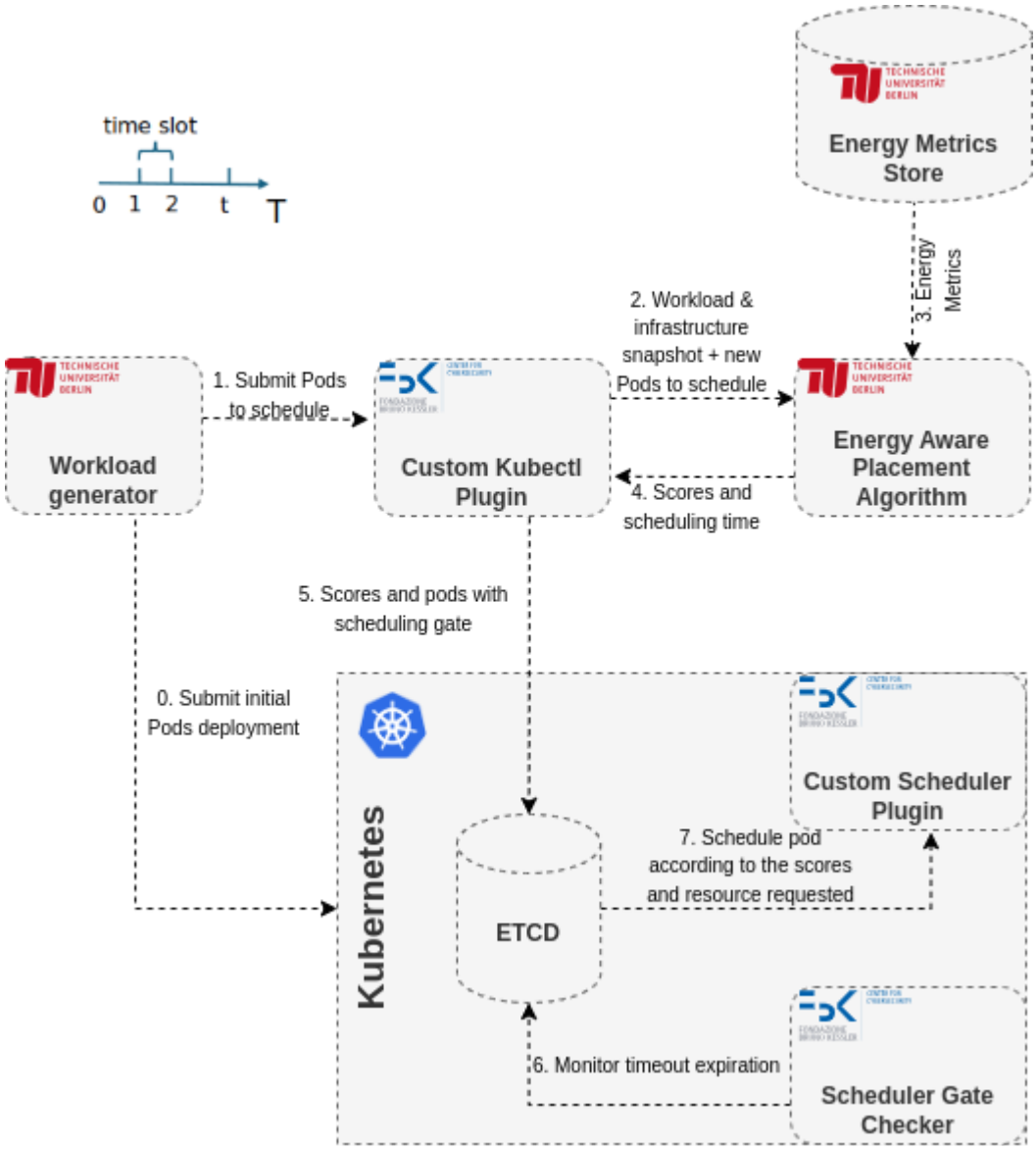
What is TUB Energy Aware Placement Algorithm

TUB `Energy Aware Placement Algorithm` is an algorithm able to determine *where* to place a given workload (in which K8s node) and *when* to place it (at what time to schedule it) according to the carbon footprint of each node of a K8s cluster.

Architecture of K8s Energy Aware Scheduler

High level components and workflow

The following figure shows the high level components of the integrated solution together with the workflow underwent by the Pods entering the system.



Here a brief description of the workflow.

1. `Workload generator` (TUB in our case but generally a `User` of the system) decides an initial deployment condition (i.e. a number of Pods with a given set of `resources requested` and `termination time`) that is deployed through K8s vanilla (i.e. not through the TUB algorithm) on the K8s cluster. The termination time of all the Pods submitted must be known a priori (e.g. all pods with a given label have a lifetime of 1 hour) (0.)
2. `Workload generator` (TUB in our case but generally a `User` of the system) submits one or more Pods (again with given `resources requested` and `termination time`) to the K8s cluster using the `Custom kubectl plugin`. Also in this case, the termination time of all the Pods submitted must be known a priori (1.)
3. The `Custom kubectl plugin` sends to the `Energy Aware Algorithm` information about i) the Pods to be deployed, ii) the Pods already running on the system, iii) the Pods in Pending state due to a scheduling gate, iv) the Pods in Pending state due to resource unavailability (if any) and v) the status of in infrastructures in terms of available resources (cpu, memory) for each node of the cluster (2.)
4. `Energy Aware Algorithm` collects info received and, using also metrics coming from an `Energy Metrics Store` (3.) determines the placement scores for that Pod and the time when it should be scheduled (4.)

5. `Custom kubectl plugin` creates the Pod and the related scores inside K8s cluster applying a `Scheduling Gate` until the scheduled time for that Pod (5.)
6. Through a simple cronjob, called `Scheduler Gate Checker`, the scheduled time is monitored and, once occurred, the `Scheduling Gate` is dropped and the scheduling phase is started (6.)
7. During the scheduling phase the `Custom Scheduler Plugin` is invoked by the K8s Scheduler so as to handle the custom scores imposed (7.).

The current (it should be modified to add at least the time of scheduling) specification of the `grpc IDL` is provided here.

Assumptions

A number of assumptions must be established to implement the solution described in the previous section:

1. We consider K8s nodes, not FluidOS nodes. However, if needed, we can split K8s nodes in different geographical/logical regions.
2. The `termination time` of all the Pods submitted must be known a priori (e.g. all pods with a given label have a lifetime of 1 hour).
3. In order to reduce the execution time, we can reason in terms of time slots (instead of real time): the time variables (i.e. `termination time`, `scheduling time`) can be reduced by dividing them by a constant value representing the duration of a time slot (e.g. if `termination_time=3600s` and `time_slot_duration=30s` then we reduce the experimentation to 120 time slots or 120s)
4. Scores should be given by the algorithm per K8s node and per Pod (in case of more than one replica, also per replica, but we would avoid to considering more than one replica)
5. K8s relies on the availability of the `resources requested` (cpu, memory) at the time of scheduling, not on `resource utilization`. Therefore, any prediction of resource utilization could be useless because K8s bases its scheduling decision on resource requested instead. A suggestion is to consider resource utilization constant and always equal to resource requested. We assume that the resources requested are available on the node, selected for hosting the Pod, when scheduling is triggered
6. FBK `Custom kubectl plugin` cannot provide any energy metric
7. In order to test the effectiveness of the solution against K8s vanilla, some KPIs should be defined. Moreover, the comparison against an "optimal solution" is currently not possible because we do not have an optimal solution.
8. If we need to delete pods, we can do so in a `kwok` emulated environment using `Stages`. But we need to know a priori the lifetime duration of each Pod.